

test_summarize.py

```
1 import json
2 from pathlib import Path
3
4 import httpx
5 import pytest
6
7 BASE_URL = "http://127.0.0.1:8000"
8
9 PROJECT_ROOT = Path(__file__).resolve().parents[1]
10 LONG_TEXT_PATH = PROJECT_ROOT / "project_inputs" / "Long_text.txt"
11 FORBIDDEN_PATH = PROJECT_ROOT / "project_inputs" / "forbidden_examples.json"
12
13
14 def load_forbidden_cases():
15     content = json.loads(FORBIDDEN_PATH.read_text(encoding="utf-8"))
16     cases = []
17     for category, examples in content.items():
18         for index, example in enumerate(examples):
19             cases.append(
20                 pytest.param(
21                     example["input"],
22                     example["policy_code"],
23                     id=f"{category}-{index}",
24                 )
25             )
26     return cases
27
28
29 def test_health_endpoint():
30     r = httpx.get(f"{BASE_URL}/health")
31     assert r.status_code == 200
32     assert r.json()["status"] == "ok"
33
34
35 def test_summarize_valid_long_text():
36     long_text = LONG_TEXT_PATH.read_text(encoding="utf-8")
37     payload = {
38         "text": long_text,
39         "source_id": "valid-document-001",
40     }
41     r = httpx.post(f"{BASE_URL}/summarize", json=payload)
42     assert r.status_code == 200
43     data = r.json()
44
45     assert isinstance(data["summary"], str)
46     assert data["summary"].strip()
47     assert len(data["summary"]) < len(long_text)
48     assert data["refused"] is False
49
50
51 def test_response_shape_schema_fields():
52     payload = {"text": "This is a short test document.", "source_id": "doc-456"}
53     r = httpx.post(f"{BASE_URL}/summarize", json=payload)
54     assert r.status_code == 200
55     data = r.json()
56
57     assert "summary" in data
58     assert "word_count" in data
59     assert "source_id" in data
60     assert "refused" in data
```

```

61     assert "policy_code" in data
62     assert "message" in data
63
64     assert isinstance(data["summary"], str)
65     assert isinstance(data["word_count"], int)
66     assert isinstance(data["source_id"], str)
67     assert isinstance(data["refused"], bool)
68
69     assert data["source_id"] == payload["source_id"]
70     assert data["word_count"] >= 0
71     assert data["word_count"] == len(data["summary"].split())
72     assert data["refused"] is False
73     assert data["policy_code"] is None
74     assert data["message"] is None
75
76
77 @pytest.mark.parametrize("unsafe_text,expected_policy_code", load_forbidden_cases())
78 def test_forbidden_content_is_refused(unsafe_text, expected_policy_code):
79     payload = {"text": unsafe_text, "source_id": "unsafe-doc-001"}
80     r = httpx.post(f"{BASE_URL}/summarize", json=payload)
81     assert r.status_code == 200
82     data = r.json()
83
84     assert data["summary"] == ""
85     assert data["word_count"] == 0
86     assert data["source_id"] == payload["source_id"]
87     assert data["refused"] is True
88     assert data["policy_code"] == expected_policy_code
89     assert data["message"] == "refused: policy_violation"
90     assert unsafe_text not in data["summary"]
91
92
93 def test_policies_endpoint_returns_supported_policies():
94     response = httpx.get(f"{BASE_URL}/policies")
95     assert response.status_code == 200
96     data = response.json()
97     assert "policies" in data
98     assert isinstance(data["policies"], list)

```